

SISTEMAS DE TIEMPO REAL

Concurrencia – Paralelismo - SOTR

Definiciones

Concurrencia: “Coincidencia, concurso simultáneo de varias circunstancias (RAE)”

- En el contexto de la programación, la concurrencia es la capacidad de que **algunas de las tareas** que resuelven un **problema** se puedan realizar en **paralelo**.
- Está presente en la naturaleza, la vida diaria, los sistemas de cómputo, etc.

Definiciones

Concurrencia: “Coincidencia, concurso simultáneo de varias circunstancias (RAE)”

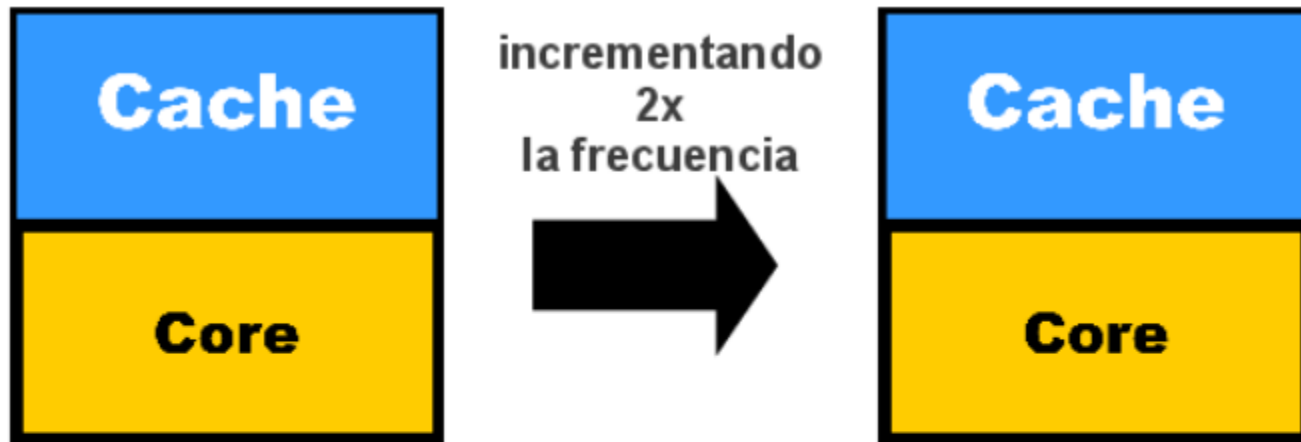
- En el contexto de la programación, la concurrencia es la capacidad de que **algunas de las tareas** que resuelven un **problema** se puedan realizar en **paralelo**.
- Está presente en la naturaleza, la vida diaria, los sistemas de cómputo, etc.
- ¿Qué significa que pueden ser realizadas en paralelo?

Definiciones

Paralelo: “A la vez, a un mismo tiempo, simultáneamente(RAE)”

- En el contexto de la programación, el paralelismo es la condición en que **dos o más tareas** que resuelven un **problema** se realizan **exáctamente al mismo tiempo**.
- Mientras la concurrencia es una característica de las tareas, el paralelismo se asocia a los recursos que se disponen para poder realizar esas tareas simultáneamente.

Hardware



Frecuencia:

Potencia:

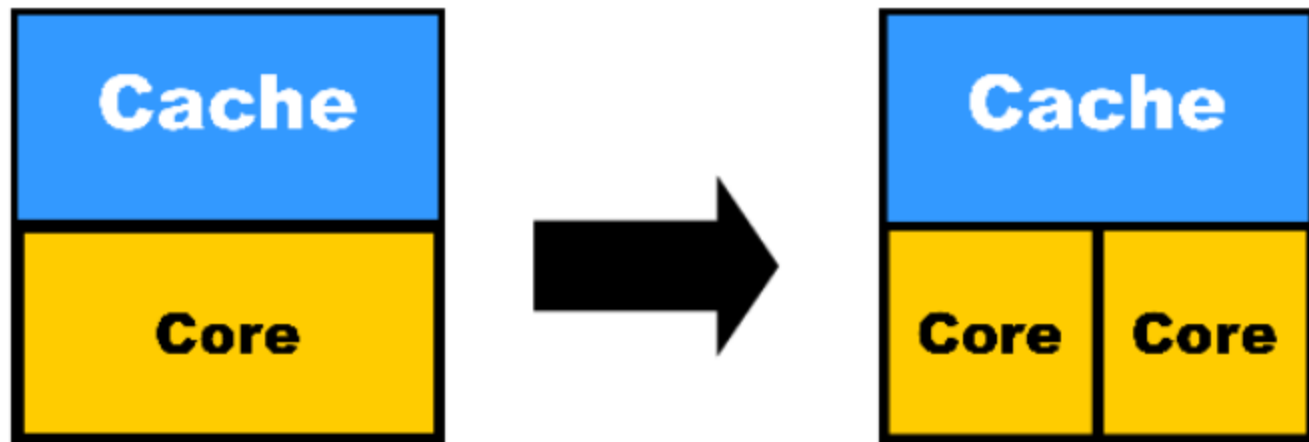
Rendimiento:

Frecuencia:

Potencia:

Rendimiento:

Hardware



Frecuencia:

Potencia:

Rendimiento:

Frecuencia:

Potencia:

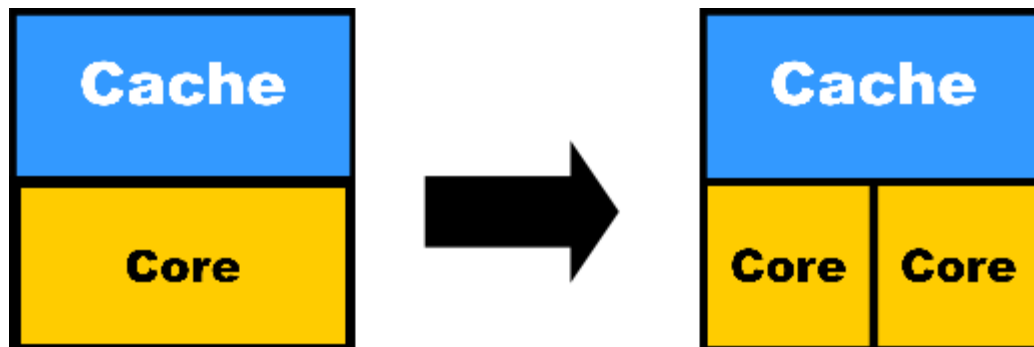
Rendimiento:

Hardware

Ventajas

- Se puede compartir recursos
- Se puede escalar en rendimiento ¿simplemente?
Aumentando el número de núcleos de procesamiento
- Menos consumo

¿es directa la relación entre número de cores y rendimiento?



Hardware

Observaciones

- Duplicar recursos no siempre conduce a que las cosas funcionen más rápido.
- Es necesario especificar de qué manera se divide el trabajo para aprovechar los recursos disponibles.
- **Para obtener el máximo rendimiento de la arquitectura es necesario recurrir a la programación concurrente**

Software

Concurrencia

Cualquier sistema más o menos “inteligente” exhibe concurrencia:

- Desde un teléfono hasta un automóvil
- Navegador Web accediendo información mientras atiende al usuario
- Acceso de varias aplicaciones a disco
- Varios usuarios conectados al mismo sistema (ej, haciendo una reserva)
- Juegos
- ¿Otros?

Software

Concurrencia

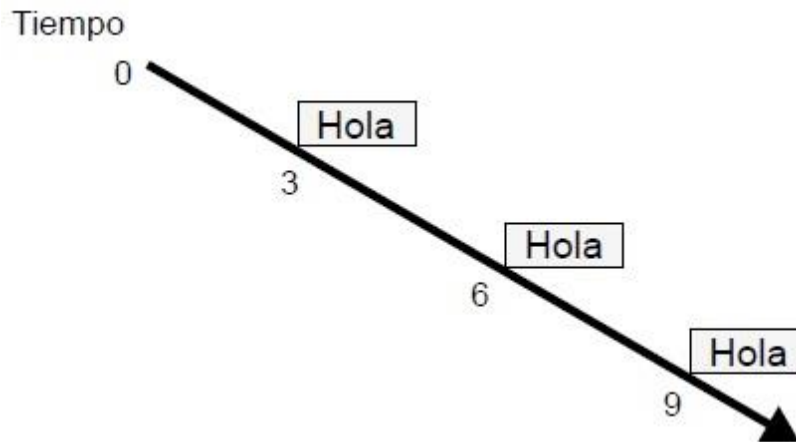
- Mejor estructura de las aplicaciones (es más apropiado programar múltiples actividades independientes y concurrentes)
- Reacción a entradas asincrónicas (ej: sensores)
- Mejora en la respuesta: no bloquear la aplicación completa por E/S, incrementar el rendimiento de la aplicación por mejor uso del hardware
- Sistemas distribuidos:
 - Una aplicación en varias máquinas
 - Sistemas C/S o P2P

La necesidad de sistemas de cómputo cada vez más poderosos y flexibles atenta contra la simplificación de asunciones de secuencialidad

Software

Ejemplo

Problema: Desplegar cada 3 segundos el mensaje “Hola”
Reacción a entradas asincrónicas (ej: sensores)

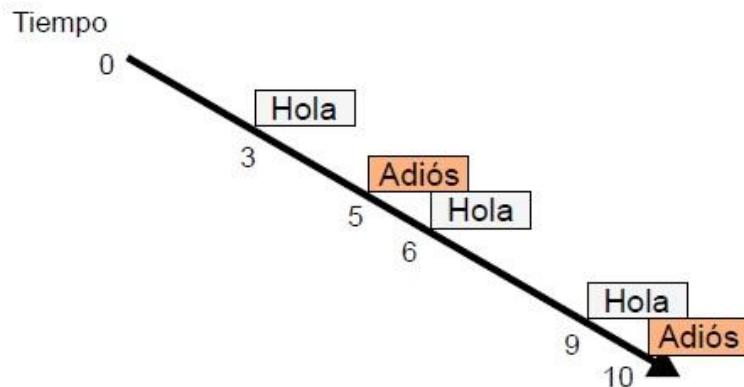


```
1  while (true){  
2      sleep(3);  
3      show("Hola");  
4  }
```

- ¿Qué ocurre si se quiere mostrar, además, otro mensaje “Adiós” pero cada 5 segundos?

Software

Solución secuencial



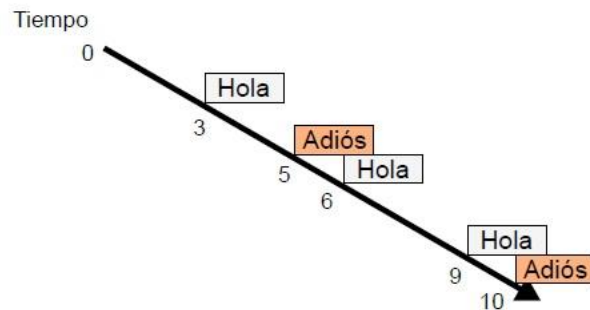
```
1  int t = 0;
2  while (true){
3      sleep (1);
4      t = t + 1;
5      if (t % 3 == 0)
6          show ("Hola");
7      if (t % 5 == 0)
8          show ("Adios");
9  }
```

- El programa (secuencial) es más complejo. . .

Software

Solución concurrente

Una solución más natural es ejecutar dos algoritmos simples concurrentemente. . .



```
1  PROCESS Msj1(){
2      while (true){
3          sleep (3);
4          show ("Hola");
5      }
6  }
7
8  PROCESS Msj2(){
9      while (true){
10         sleep (5);
11         show ("Adios");
12     }
13 }
```

Opciones

- Solución secuencial
- Solución concurrente
- Solución paralela

Solución secuencial

Programación secuencial

- Un programa secuencial consiste en una secuencia de sentencias que son ejecutadas una detrás de la otra. En todo momento existe sólo un *thread* de control.
- El ensamblado final del objeto sólo se podrá realizar luego de N pasos de procesamiento o fabricación. **Nos fuerza** a establecer un **estricto orden temporal**.

Solución concurrente

Programación concurrente

Un programa concurrente contiene dos o más procesos que realizan una tarea en conjunto. Cada proceso es un programa secuencial. En un programa concurrente, cada proceso tiene su propio *thread* de control.

Disponemos de N procesos, donde cada uno puede trabajar al mismo tiempo en una parte diferente.

- **Consecuencias**

- Menor tiempo para completar el trabajo
- Menor esfuerzo individual
- Paralelismo del hardware

- **Dificultades**

- Necesidad de compartir recursos evitando conflictos
- Necesidad de esperarse en puntos clave
- Necesidad de comunicarse

Solución paralela

Programación paralela

Resolver el problema en el menor tiempo usando una arquitectura multiprocesador en la que se pueda distribuir la tarea global en tareas que puedan ejecutarse en procesadores simultaneamente.

¿Cómo se mapean los procesos en procesadores?

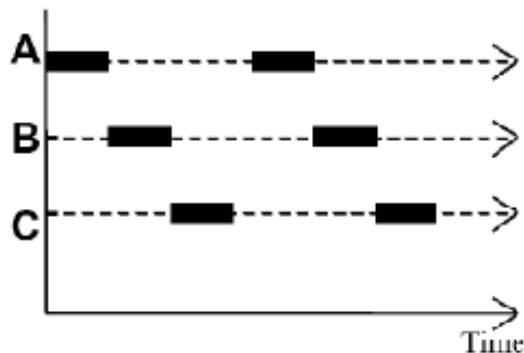
- Si disponemos de M procesadores ($M=N$), uno para cada proceso
- Si disponemos de 1 procesador, dedicar parte del tiempo a cada parte del objeto (*Concurrencia sin paralelismo de hard*).
- ¿ $M < > N$?

CONCURRENCIA $\Rightarrow$ *Concepto de software* no restringido a una arquitectura particular de hardware ni a un número determinado de procesadores

Concurrencia vs Paralelismo

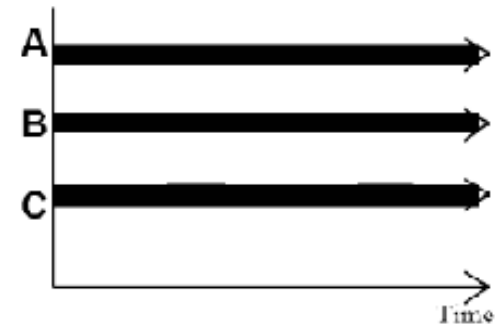
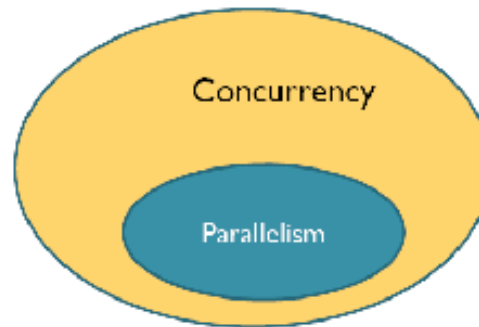
- **Concurrencia “interleaved”** ⇒

- Procesamiento simultáneo lógicamente
- Ejecución intercalada en un único procesador
- “Seudo-paralelismo”



- **Concurrencia simultánea** ⇒

- Procesamiento simultáneo físicamente
- Requiere un sistema multiprocesador o multicore
- Paralelismo “full”



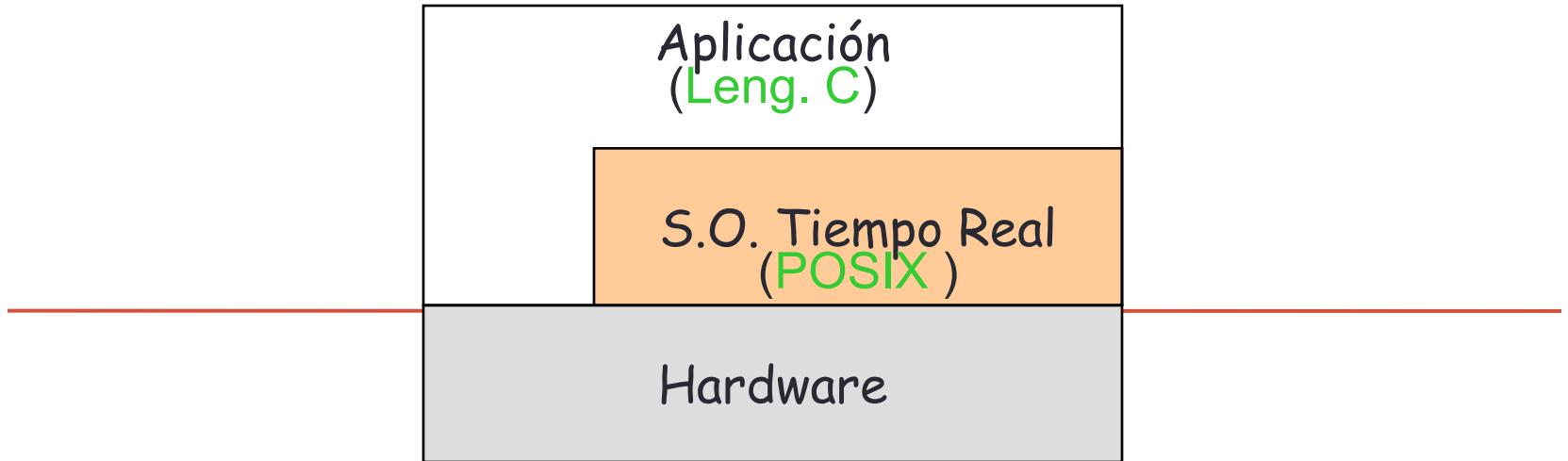
Soporte en S. O.

Todos los sistemas operativos soportan procesos:

- Cada proceso se ejecuta en una máquina virtual distinta. Algunos sistemas operativos soportan procesos ligeros (hilos o threads):
 - Proceso “liviano” que tiene su propio contador de programa y su pila de ejecución, pero no controla el “contexto pesado” (por ejemplo, las tablas de página).
 - Todos los hilos de un proceso comparten la misma máquina virtual
 - Tienen acceso al mismo espacio de memoria
 - El programador o el lenguaje deben proporcionar mecanismos para evitar interferencias
- La concurrencia puede estar soportada por:
 - el lenguaje: Java, Ada, occam2
 - el sistema operativo: C/POSIX

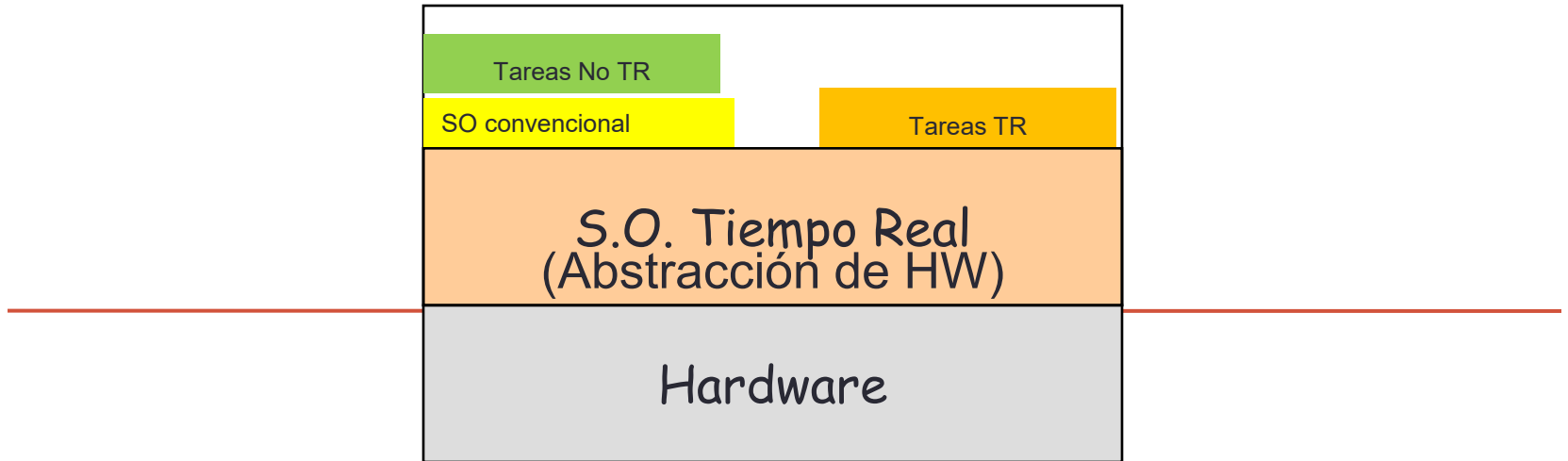
SISTEMAS OPERATIVOS DE TIEMPO REAL

Entornos de ejecución



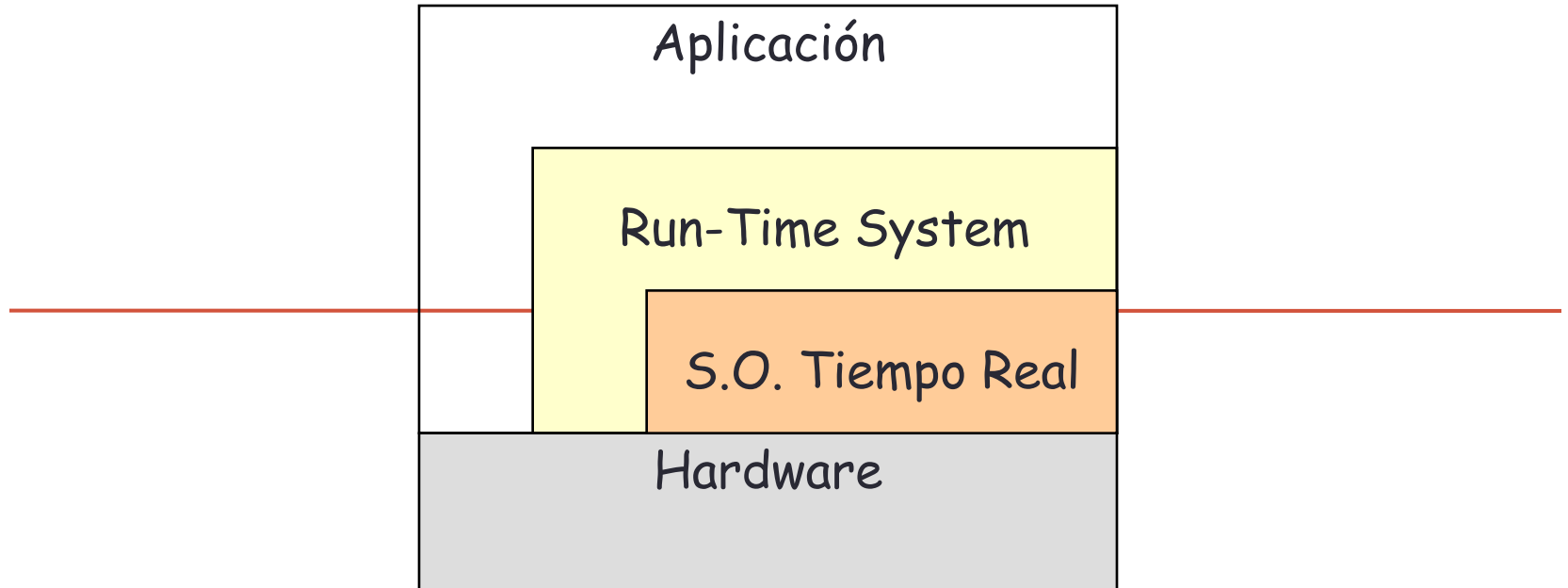
Ejemplo: QNX, Linux 2.6 parche RT

Entornos de ejecución



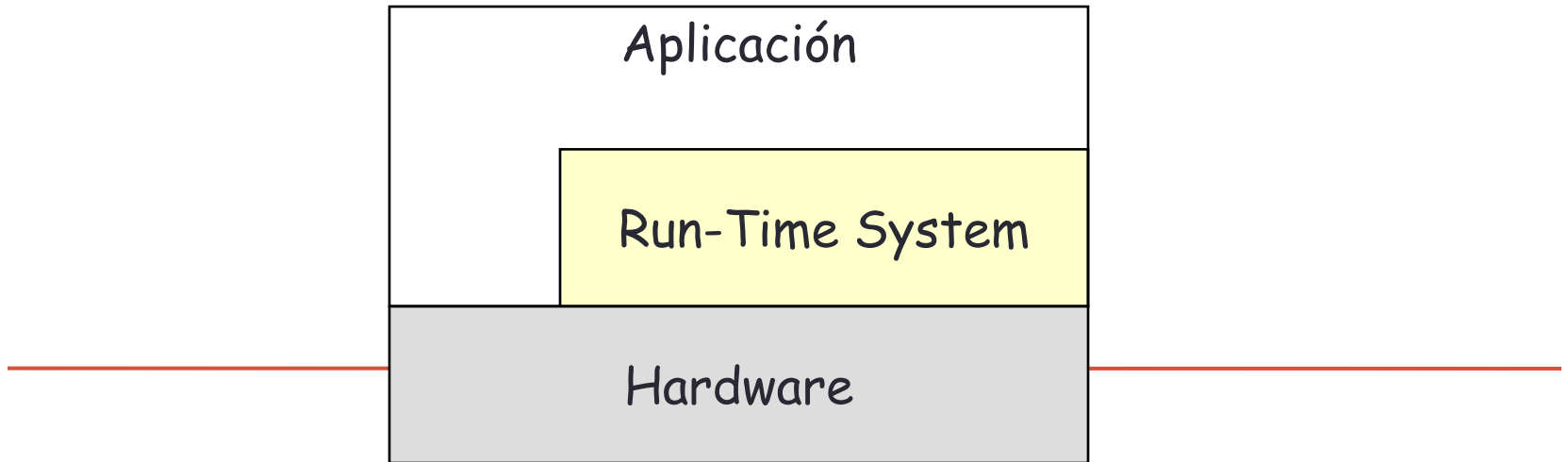
Ejemplo: RTLinux, RTAI

Entornos de ejecución



Ejemplo: ADA

Entornos de ejecución



Ejemplo: MaRTEOS

Características

- ❑ Los Sistemas Operativos Embebidos se caracterizan por:
 - ❑ Tamaño mínimo.
 - ❑ Rápidos cambios de contexto.
 - ❑ Diseñados para ser portables a varias arquitecturas.
 - ❑ Están diseñados para hardware orientado a la entrada/salida, mas que al procesamiento.
 - ❑ Bajo intercambio de datos entre memoria principal y almacenamiento secundario.

Clasificación

❑ Estáticos

- Las tareas, sus recursos, los eventos y las interrupciones, se definen en tiempo de compilación conjunto con la alocaación de memoria.

❑ Dinámicos

- La alocaación de memoria, creación de tareas y estructuras de control se dan en tiempo de ejecución.

FREERTOS

Características

- ❑ Código Abierto
- ❑ Mayoría de archivos en C
- ❑ Amplia comunidad de desarrolladores
- ❑ Portado a mas de 35 arquitecturas
- ❑ Mucha documentación
- ❑ Probado correctamente en arquitecturas entre 8KB y 512KB de memoria Flash y 16KB a 256KB de memoria RAM
- ❑ Generalmente un binario ya compilado se llevará entre 4 KB y 9 KB

Características

- ❑ Asignación de prioridades flexible
- ❑ Manejo de colas, para el pasaje de mensaje entre tareas
- ❑ Provee semáforos binarios, de conteo, recursivos y de exclusión mutua
- ❑ Funciones para el control del planificador:
 - ❑ Tick de reloj
 - ❑ Inactividad
 - ❑ Chequeo de desbordamiento de pila

Modos de Funcionamiento

- ☐ Cooperativo
- ☐ Preemptive
- ☐ Mixto

Configuración

- FreeRTOSConfig.h
- configUSE_PREEMPTION
 - Se configura con 1 para que funcione como un RTOS preventivo y con 0 como un RTOS cooperativo
- configCPU_CLOCK_HZ
 - Se configura la velocidad del reloj del sistema
- configTICK_RATE_HZ
 - Se configura cada cuanto se desea que se genere un tick de sistema
- configMAX_PRIORITIES
 - Numero maximo de prioridades a utilizar

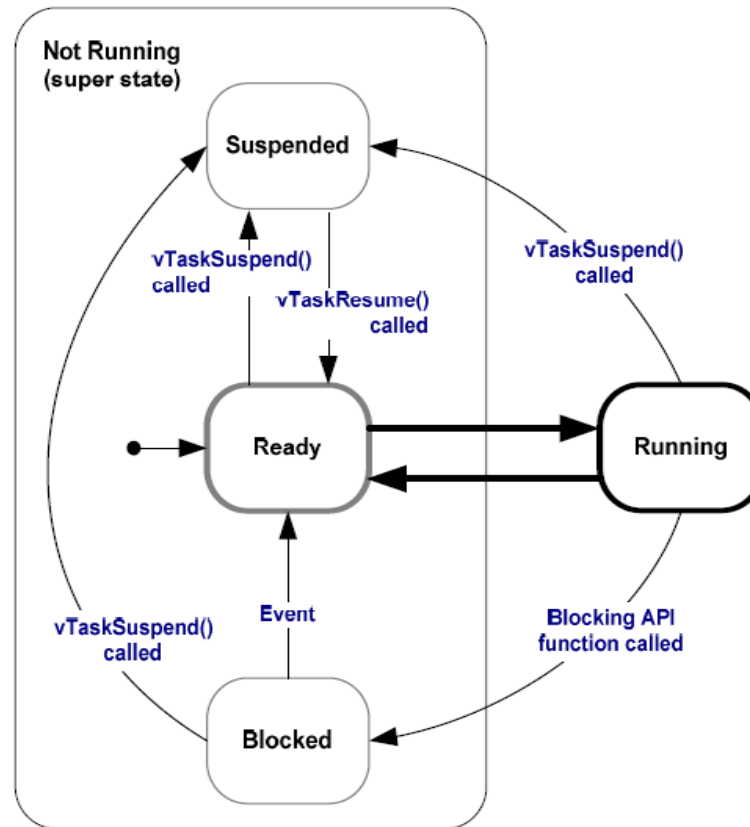
Configuración

- FreeRTOSConfig.h
- configMINIMAL_STACK_SIZE
Tamaño de la pila minimo
- configTOTAL_HEAP_SIZE
 - Tamaño de la cola de memoria disponible para el RTOS
- configUSE_USE_MUTEXES
 - Si se setea en 1 se utilizan MUTEXES
- configUSE_CO_ROUTINES
 - Si se setea en 1 se utilizaran corutinas (debe incluirse la source corutine.c) para manejo cooperativo
- #define INCLUDE_vTaskDelete 1
 - Aqui se incluye en el proceso de build a la tarea "borrar Tarea"

Tareas

❑ xTaskCreate	Iniciar una tarea
❑ vTaskDelete	Finalizar una tarea
❑ vTaskDelay	Detener una tarea un tiempo determinado
❑ vTaskPrioritySet	Asignar la prioridad de una tarea
❑ uxTaskPriorityGet	Obtener la prioridad de una tarea
❑ vTaskSuspend	Suspender la ejecución de una tarea
❑ vTaskResume	Resumir la ejecución de una tarea

Tareas



Tareas – Programa Principal

```
int main(void)
{
    prvSetupHardware();
    xTaskCreate(Tarea1 , "Tarea 1", 1000, null, 1 , null);
    xTaskCreate(Tarea2, "Tarea 2", 1000, null, 1 , null);
    vTaskStartScheduler();
    return 1;
}
```

Tareas - Create

- xTaskCreate(Tarea1 , “Tarea 1”, 1000, null, 1 , null)
 - ❑ Tarea1 : Puntero de la función que implementa la tarea
 - ❑ “Tarea 1”: Nombre de la tarea (identificador)
 - ❑ 1000: Tamaño de la pila de la tarea
 - ❑ null: Puntero a los parámetros que se le pasan a la tarea
 - ❑ 1: Prioridad de la tarea (0 = Minima prioridad)
 - ❑ null: Manejador de la tarea

Herramientas de Sincronización

Semáforos

- ❑ Semáforos Binarios
- ❑ Semáforos Contadores
 - ❑ xSemaphoreCreateBinary
 - ❑ xSemaphoreCreateCounting
 - ❑ vSemaphoreDelete

Herramientas de Sincronización

Semáforos Binarios

- ❑ `xSemaphoreTake(sem, timeout)`

Toma el Semáforo para proteger el recurso compartido

- ❑ `xSemaphoreGive(sem)`

Libera el Semáforo y deja el acceso al recurso para las demás tareas

Una tarea puede tomar el semáforo “take”, pero es otra la tarea que se encarga de liberarlo “give”, sirve para un control por eventos.

Herramientas de Sincronización

Colas

- ❑ `xQueueCreate` Crea una cola, especificando cantidad y tamaño de ítems
- ❑ `xQueueSend` Pone un nuevo dato en la cola
- ❑ `xQueueReceive` Saca un dato de la cola
- ❑ `xQueuePeek` Lee un dato de la cola, pero no lo saca
- ❑ `uxQueueSpacesAvailable` Devuelve la cantidad de lugares libres en la cola